



NorthGold

Technical Reference

How the platform decides what to pay, who verifies what, and how commission moves through the network — plus how to run and use all three builds.

COVERS

Django API · Next.js user app · Next.js admin panel · Vite demo

CURRENCY

US Dollar (\$), `en-US` grouping throughout

REPOSITORY

github.com/ShivamKumar-123/NorthGold

AUDIENCE

Developers running, deploying or extending the platform

Contents

- 1 What the platform does
- 2 The three builds, and which to run
- 3 Core logic — the rules money obeys
- 4 ROI plans and the payout engine
- 5 Deposits, withdrawals and the ledger
- 6 Referral commission (MLM)
- 7 Running each build
- 8 Using it — member walkthrough
- 9 Using it — administrator walkthrough
- 10 Admin configuration reference
- 11 Deploying the demo
- 12 Limits and what is not production-grade

How to read this

Sections 3 to 6 are the logic: what the system computes and why. Sections 7 to 10 are operational: how to start it and what each screen is for. If you only need to demo the product, read section 7 (demo), then 8 and 9.

One idea underpins everything here

A member is promised a specific schedule of payments at the moment they invest. Every rule in this document exists to make sure that promise is kept exactly — not approximately, not generously, and not retroactively changed.

1 · What the platform does

NorthGold takes a deposit, places it in a fixed-income plan, and pays a contracted percentage back every month for the term. People who introduce other members earn commission on what those members deposit and on what they are subsequently paid.

The cycle in one page

- 1 **A member deposits cash** at the counter and files a request describing the handover — who took it, where, when, and the receipt number.
- 2 **An administrator verifies it** against the counter receipt and approves. Nothing moves until this happens.
- 3 **The wallet is credited** and, if auto-invest is on, an investment opens immediately on the plan whose slab contains that amount.
- 4 **Commission cascades** up the depositor's sponsor chain, level by level, paying each upline its configured share of the deposit.
- 5 **Every month the schedule pays out.** The percentage for that month is applied to the original principal and credited to the wallet — and commission cascades again on that payout.
- 6 **At maturity the principal returns** to the wallet and the investment closes.
- 7 **The member withdraws** by requesting a cash collection; the amount is held immediately and handed over once an administrator confirms.

Why cash only

Every movement is a physical hand-over confirmed by a person. The data model still contains bank, UPI and crypto methods, and the code paths exist — but nothing in the product offers them, so there is exactly one way money enters and leaves, and exactly one thing to verify.

2 · The three builds, and which to run

BUILD	STACK	PORT	USE IT WHEN
backend/	Django · DRF · Channels · Celery	8000	The real system. Holds the database and every rule.
frontend/user-app/	Next.js 15 (App Router)	3000	What members see. Needs the backend running.
frontend/admin-app/	Next.js 15 (App Router)	3001	Staff panel. Needs the backend running.
demoWeb/	Vite · React · React Router	5173	Showing the product with nothing to install or host.

The demo is not a mock-up

`demoWeb` reimplements every rule in this document against `localStorage`. Its seed does not write plausible-looking balances — it **replays the real actions** (deposit, approve, auto-invest, cascade commission, run every payout since due), which is why the ledger adds up and each screen agrees with the others. It also contains the admin panel, at `/admin`, rather than as a second app.

Which one to open

To *demo* the product: `demoWeb` alone — no backend, no database, no configuration. To *develop* the real system: backend first, then the two Next apps.

What the demo does differently

	REAL SYSTEM	DEMO
Storage	PostgreSQL / SQLite via DRF	<code>localStorage</code> , per browser
Admin	Separate app on :3001	<code>/admin</code> routes in the same app
Prices	WebSocket feed	A timer nudging feed-priced rows $\pm 0.4\%$
Payouts	Celery beat, scheduled	Caught up whenever the app loads
Uploads	Stored files	File <i>name</i> only — see §12

3 · Core logic — the rules money obeys

Six rules explain almost every behaviour in the system. Each exists because the obvious alternative is quietly wrong.

1. The schedule is frozen at purchase

An investment **copies** the plan's month-by-month curve when it opens. Editing a plan afterwards changes what future investments are promised and never touches an existing one.

Why

A member read a specific schedule before committing. If the plan were referenced rather than copied, an administrator adjusting a rate would silently rewrite what everyone already holding that plan was promised.

2. Money rounds down, always

Every payout and every commission is floored to two decimals, never rounded to nearest.

Why

Rounding half-up gains a fraction of a cent on roughly half of all payments. Across twelve months and five commission levels, that is a systematic over-payment against a schedule that promised an exact figure.

3. Payout runs are idempotent

Each investment carries a `months_paid` cursor. The engine pays only months past that cursor whose due date has arrived, then advances it.

Why

In the real system Celery can retry a task; in the demo the engine runs on *every* page load. Without a cursor, either would double-pay.

4. Due dates clamp to the end of the month

A deposit on the 31st pays on the 28th in February, not on the 3rd of March.

5. Withdrawals hold funds at request time

The balance leaves the wallet the moment a withdrawal is requested, not when it is approved. Rejection returns it.

Why

Holding at approval would let the same balance back two pending withdrawals at once, and the second would be approved against money that no longer exists.

6. The ledger is append-only

Every movement writes a transaction row recording the amount, the resulting balance and a human-readable note. Balances are never edited in place without a row explaining the change.

4 · ROI plans and the payout engine

Slabs

A deposit amount selects exactly one plan. Slabs never overlap and the top tier is open-ended, so any amount at or above the entry minimum matches one and only one plan.

PLAN	DEPOSIT RANGE	MONTH 1	MONTH 12	TOTAL OVER TERM
Starter	\$100 – \$999	1.00%	1.50%	13.90%
Silver	\$1,000 – \$4,999	1.00%	2.50%	19.75%
Gold	\$5,000 – \$24,999	1.50%	3.00%	25.00%
Platinum	\$25,000 and above	2.00%	3.50%	31.75%

All four run 12 months. The curves *ramp* — a later month pays more than an earlier one — which is the incentive to stay invested. Both the slab boundaries and every monthly percentage are editable in the admin panel.

What a month pays

```
payout(month N) = floor2( principal × percent[N] / 100 )
```

```
Example – $30,000 on Platinum, month 6 pays 2.50%  
= floor2(30000 × 2.50 / 100)  
= $750.00
```

The percentage always applies to the **original principal**. Returns are credited to the wallet as withdrawable balance and do not compound. (`roi_credit_target` can be switched to `principal` to compound instead.)

The engine, step by step

- 1 For each active investment, look at month `months_paid + 1`.
- 2 Compute its due date: start date plus that many months, clamped to month end.
- 3 If the due date is in the future, stop — this investment is up to date.
- 4 Otherwise credit the payout, write the ledger row, advance the cursor, and cascade commission on the amount.
- 5 Repeat until the schedule runs out or a future date is reached.
- 6 When the last month is paid, return the principal to the wallet and mark the investment matured.

When does it run?

Real system: a Celery beat task. Demo: on app load, because there is no scheduler in a browser — which is safe precisely because of the cursor.

5 · Deposits, withdrawals and the ledger

Deposit

- 1 Member hands cash over at the counter and collects a receipt.
- 2 They file a deposit request: amount, receipt number, and a written description of the handover. **The description is mandatory** — cash leaves no bank trail, so it is the only thing an administrator can verify against.
- 3 The request sits as *pending*. No balance changes.
- 4 An administrator approves or rejects it. A rejection requires a reason, which the member sees.
- 5 On approval: wallet credited → investment opened (if auto-invest is on) → commission cascaded.

Withdrawal

- 1 Member requests an amount and says when they will collect.
- 2 The amount is **held immediately** — it leaves the available balance and a `withdrawal_hold` row is written.
- 3 An administrator approves (records the hand-over) or rejects (returns the money with a `withdrawal_refund` row).

Transaction kinds

KIND	EFFECT ON WALLET	WRITTEN WHEN
<code>deposit</code>	+	An administrator approves a deposit
<code>investment</code>	–	Principal moves from wallet into an investment
<code>roi_payout</code>	+	A scheduled month falls due and is paid
<code>commission</code>	+	A downline's deposit or payout pays this member
<code>principal_return</code>	+	An investment matures
<code>withdrawal_hold</code>	–	A withdrawal is requested
<code>withdrawal_refund</code>	+	A withdrawal is rejected
<code>withdrawal</code>	0	Approved — records the hand-over; the hold already moved the money

Reading the balances

Available is what can be invested or withdrawn right now. *Locked in investments* is principal currently earning. A pending withdrawal is in neither — it has been held.

6 · Referral commission

Everyone joining through a referral link is placed under the person whose code they used. Commission then walks *up* that chain from whoever triggered it.

The ladder

LEVEL	RELATIONSHIP	ON THEIR DEPOSIT	ON THEIR MONTHLY RETURN	UNLOCKS AT
L1	Someone you personally introduced	5.0%	10.0%	0 directs
L2	Their introductions	3.0%	5.0%	1 direct
L3	One level deeper	2.0%	3.0%	2 directs
L4		1.0%	2.0%	3 directs
L5		0.5%	1.0%	4 directs

How a cascade works

- 1 Something pays out — a deposit is approved, or a monthly return is credited.
- 2 Start at the depositor's sponsor. That is level 1.
- 3 Check the unlock rule: does this person have at least `min_directs` direct referrals? If not, record a *skip* with the reason and move on without paying.
- 4 Otherwise pay `floor2(base × percent / 100)`, credit the wallet, write the ledger row.
- 5 Move to that person's sponsor — level 2 — and repeat, up to the configured maximum depth (5 by default).
- 6 Stop at the top of the chain, at the depth limit, or if a sponsor loop is detected.

Worked example

Chain: Anita ← Rahul ← Priya (Priya is at the top)

Event: Anita's \$900 deposit is approved

L1 Rahul 5.0% of 900 = \$45.00 → paid

L2 Priya 3.0% of 900 = \$27.00 → paid if Priya has ≥1 direct

Later, Anita's month-1 return of \$9.00 is credited:

L1 Rahul 10.0% of 9 = \$0.90 → paid

L2 Priya 5.0% of 9 = \$0.45 → paid

Skipped payments are recorded, not discarded

A level that paid nothing because it was not unlocked still writes a row with its reason. The member sees it in their commission log — which is what turns "why did I not get paid?" into an answer instead of a support ticket.

Two switches worth knowing

- `mlm_deposit_enabled` — pay commission when a deposit is approved.
- `mlm_roi_enabled` — pay commission on every monthly return. This is the larger of the two over a full term.

7 · Running each build

The demo — nothing to install beyond npm

```
cd demoWeb
npm install
npm run dev          # http://localhost:5173

npm run build        # production build into dist/
npm run preview      # serve that build on :4173
```

No backend, no database, no environment variables. Open and use.

Demo accounts

ROLE	EMAIL	PASSWORD
Member	priya@northgold.demo	demo1234
Administrator	admin@northgold.demo	admin123

Both sign-in screens carry a *click to fill* button. Every seeded member uses `demo1234`: priya, arjun, rahul, sneha, neha, vikram and anita, all `@northgold.demo`. The admin panel is at `/admin`.

The real system

```
# 1 · API
cd backend
python -m venv .venv && .venv\Scripts\activate    # Windows
pip install -r requirements.txt
copy .env.example .env                            # then edit SECRET_KEY
python manage.py migrate
python manage.py seed_platform \
    --admin-email you@example.com --admin-password 'choose-one'
python manage.py runserver 8000

# 2 · Member app                                # 3 · Admin panel
cd frontend/user-app                            cd frontend/admin-app
npm install                                     npm install
copy .env.local.example .env.local              copy .env.local.example .env.local
npm run dev      # :3000                        npm run dev      # :3001
```

There are no default administrator credentials

The seed command only creates one if you pass `--admin-email` and `--admin-password`. That is deliberate — a shipped default password is a shipped vulnerability.

Verifying the rules still hold

```
cd backend  
python manage.py test      # 35 tests
```

The suite covers the payout schedule, rounding, idempotency, the commission cascade with its unlock rules, and the downline tree including its depth boundary.

8 · Using it — member walkthrough

The fastest way to understand the product is to run one deposit end to end and watch it reach four different screens.

- 1 **Open the landing page.** The plan cards show each slab and its month curve. The calculator computes a full schedule for any amount — using the same code that will later pay it, not a second implementation.
- 2 **Sign in** (or open an account; a referral code is optional and an unknown one is ignored rather than blocking the signup).
- 3 **Dashboard.** Available balance, active principal, lifetime returns and referral earnings, the next payouts due, and your network as a graph.
- 4 **Wallet → Deposit.** Enter an amount, the receipt number, and a description of the handover. Submit. It appears as *pending* and nothing has moved yet.
- 5 **Wait for approval** (see §9 — in the demo you can do this yourself). Once approved: the wallet is credited, an investment opens, and your sponsor chain is paid.
- 6 **Investments.** Each investment shows its progress through the term, what it has returned, and its full payout schedule.
- 7 **My Network.** Your referral link, the tree beneath you, what each member has paid you, a level-by-level breakdown, and the commission log — including the payments that were skipped and why.
- 8 **Wallet → Withdraw.** Request an amount and say when you will collect. The balance is held immediately.

Other screens

- **Profile** — personal details, password, and KYC document upload.
- **Support** — the floating button on every signed-in page composes a message and opens it on WhatsApp. It is a composer, not a live chat: there is no inbox behind it, and a widget that silently drops messages is worse than none.
- **About / Contact** — public pages; Contact pulls its numbers from admin settings.

9 · Using it — administrator walkthrough

The panel exists to do two things well: verify money movements, and set the rules that govern them.

The daily job — verification

- 1 **Dashboard** opens with the queue: how many deposits and withdrawals are waiting and for how much.
- 2 **Deposits.** Each request shows the member, the amount, the receipt number and their description of the handover. Check it against the counter receipt.
- 3 **Approve** — and be deliberate, because this is the moment the platform takes on an obligation: it credits the wallet, opens an investment, and pays commission up the chain. Or **reject** with a reason, which the member sees.
- 4 **Withdrawals** work the same way. The money was already held when the member asked. Approving records the hand-over; rejecting returns it to their wallet.

Approval is not reversible from the panel

There is no undo. An approved deposit has already opened an investment and paid commission to several people. Verify before approving, not after.

Reading the dashboard

- **Total deposited / withdrawn** — approved lifetime figures, and the net inflow between them.
- **Active principal** — money currently earning returns, i.e. the platform's live obligation.
- **ROI paid out / commission paid out** — what has already been settled. Their sum is the total obligation met so far.
- **Commission by level** — where the referral spend is actually going. Heavy L1 with almost nothing deeper means the network is wide but shallow.

Users & network

Search by name, email or referral code. Selecting a member draws the tree beneath them. Pending KYC documents appear at the top of this screen for approval or rejection.

10 · Admin configuration reference

ROI plans

Edit slab boundaries, term length and every monthly percentage. Changing the term adds or trims months from the curve so it always has exactly one entry per month — a missing entry would make a payout read `undefined` and silently pay nothing.

Editing a plan never changes an existing investment.

Each one froze its own copy of the schedule at purchase. Edits apply to investments opened from now on.

MLM levels

Set the deposit rate, the monthly-return rate and the unlock threshold for each level. `min_directs` is the number of people that member must have personally introduced before that level pays them at all.

Settings

SETTING	WHAT IT CONTROLS
Support email / phone / hours / address	Shown on the contact page and used by the contact form.
WhatsApp number	Behind the support button. Digits only with the country code — <code>wa.me</code> rejects <code>+</code> , spaces and dashes.
Minimum deposit / withdrawal	Enforced before a request can be filed. Defaults: \$100 and \$10.
Auto-invest on deposit	An approved deposit immediately opens an investment. Off: the balance sits in the wallet until the member invests it.
Pay commission on deposits	Cascade when a deposit is approved.
Pay commission on returns	Cascade on every monthly payout. The larger of the two over a full term.
Maximum commission depth	How far up the sponsor chain a payment walks. Default 5.

Cash counter

Read-only, and deliberately so: it displays the support settings above. Duplicating the edit form here would give the same fact two places to be changed and one of them to be forgotten.

Reset demo data (demo only)

In the demo's admin sidebar and Settings screen. Reseeds the entire database from scratch — useful before showing the product to someone.

11 · Deploying the demo

`demoWeb` is a static site with no backend, so it deploys anywhere that serves files. These are the Vercel steps.

- 1 On **vercel.com**, choose *Add New* → *Project* and import the `NorthGold` repository.
- 2 **Set Root Directory to** `demoWeb`. This is the one setting that matters — the app is a subdirectory, and left at the repository root the build will fail.
- 3 Everything else is detected: framework *Vite*, build `npm run build`, output `dist`.
- 4 Add no environment variables. There is no backend to point at.
- 5 Deploy.

Check after deploying

Visit `/`, `/login` and `/admin` by typing the URL directly, then **refresh each one**. If they all load, the SPA rewrite is working.

Why that check matters

The router lives in the browser; the server has no `/admin/deposits` to serve. `demoWeb/vercel.json` rewrites everything to `index.html` so deep links survive a refresh. A catch-all is safe because Vercel resolves the filesystem before applying rewrites, so real assets are still served as themselves.

Tell visitors what a browser-local demo means

The database lives in each visitor's own browser. Two people see different data, one person's actions are invisible to another, and clearing site data resets everything. That is fine for a demo — as long as nobody expects otherwise.

12 · Limits and what is not production-grade

In the demo

- **Passwords are plain text and every check runs client-side.** Anyone can open devtools and grant themselves a balance. That is acceptable when the whole database is already in the reader's own browser — there is nothing to protect. It is not acceptable anywhere else.
- **Uploads are not kept.** Deposit proofs and KYC documents record the file name only. Base64 blobs would exhaust the ~5 MB storage quota after a handful of receipts, and there is no server to hold them.
- **The contact form composes, it does not post.** There is no enquiry inbox, so it assembles the fields into a message and hands off to WhatsApp or email. The buttons say so.

In the real system

- **Set the support WhatsApp number before going live.** It ships as a placeholder, and the support button opens a dead number until it is changed in Settings.
- **Approvals cannot be undone from the panel.** Reversing one means a manual correction, because commission has already been paid to other people.
- **Returns are not a guarantee.** The schedule is what the platform contracts to pay; it is not a prediction of investment performance. The risk notice in the footer says this, and it should stay there.

If you extend it

- Adding a settlement method means adding it to the deposit and withdrawal forms — the data model and the verification flow already handle more than cash.
- Giving the demo a real inbox means replacing one handoff in `ContactForm.tsx` with a POST, plus a model behind it.
- Changing how money is rounded, when a schedule is frozen, or when funds are held will break the tests in `backend/apps/investments/tests.py` — which is the point of them.